TestWarden

Agile Product Backlog, Sprint Plan & Team Distribution

Python + pytest AI testing platform

Product Goal

Build a platform that analyzes Python repositories, ranks risky code, generates Unit/Integration/API tests, verifies their quality through pytest and mutation testing, and requires developer approval before creating a patch or branch.

MVP Definition

The first release supports Python and pytest. It includes repository analysis, risk ranking, AI Unit Test generation, Docker execution, stability checks, mutation testing, reports, and human approval.

Priority Rules

Must Have = required for MVP. Should Have = important after MVP. Could Have = future enhancement. Story points use Fibonacci estimates: 1, 2, 3, 5, 8, 13.

Deferred Scope

Java/JUnit, IDE extension, PR comments, browser E2E, advanced performance testing, concurrency testing, automatic refactoring, Kubernetes, and full multi-language support are deferred.

Product Backlog - MVP

ID	User Story	Priority	Points
REPO-01	Connect a valid Python Git repository	Must	5
REPO-02	Clone a selected branch or commit	Must	3
REPO-03	Detect Python and pytest automatically	Must	3
REPO-04	Show clear errors for invalid or unsupported repositories	Must	2
ANALYSIS-01	Extract Python files, modules, classes, and functions	Must	5
ANALYSIS-02	Discover pytest tests, fixtures, and conftest.py	Must	3
ANALYSIS-03	Calculate coverage and complexity	Must	5
ANALYSIS-04	Analyze Git change history	Should	5
RISK-01	Rank functions by testing risk	Must	5
RISK-02	Explain the factors behind every risk score	Must	3
ROUTER-01	Recommend Unit, Integration, or API tests	Must	5
UNIT-01	Generate pytest tests for a selected function	Must	8
UNIT-02	Follow project fixtures, helpers, and naming style	Must	5
UNIT-03	Generate normal, boundary, invalid, and error cases	Must	5
ASSUMP-01	Show assumptions behind generated assertions	Must	5
SANDBOX-01	Run code and tests inside Docker	Must	8
VERIFY-01	Check syntax, imports, and pytest execution	Must	5
VERIFY-02	Rerun tests to detect instability	Must	5
VERIFY-03	Run existing tests and detect regressions	Must	5
MUT-01	Create mutations and run tests against them	Must	8
MUT-02	Calculate and display mutation score	Must	3
APP-01	Accept, reject, edit, or regenerate tests	Must	5
APP-02	Create a patch or branch only after approval	Should	5
EVAL-01	Run TestWarden on BugsInPy/Tests4Py subjects	Must	5
EVAL-02	Measure bug detection rate and coverage improvement	Must	5

Sprint Plan

Assumption: 2-week sprints. Each sprint should end with a demonstrable increment and a short review.

Sprint	Scope	Deliverable
Sprint 0 - Planning	Finalize scope, architecture, data contracts, repositories for evaluation, Git workflow, and definition of done.	SRS, architecture, backlog, repo list
Sprint 1 - Foundation	Create FastAPI project, PostgreSQL models, repository cloning, Python/pytest detection, and a basic Docker runner.	Repository can be connected and executed
Sprint 2 - Repository Analysis	Implement AST parser, pytest discovery, fixture detection, coverage collection, complexity analysis, and project structure report.	Analysis report for a Python project
Sprint 3 - Risk Ranking	Implement risk formula, high-risk target list, score explanation, deterministic ordering, and target selection.	Ranked risky functions
Sprint 4 - AI Unit Agent	Build context builder, LLM integration, pytest generation, project-style matching, edge cases, and assumption extraction.	Generated tests for selected functions
Sprint 5 - Verification	Add compile check, pytest execution, logs, repeated runs, failure reasons, and regression checking.	Verified/Rejected test report
Sprint 6 - Mutation Testing	Add mutation generation, mutant execution, killed/survived results, mutation score, and high-risk-only execution.	Mutation-validated tests
Sprint 7 - Integration and API	Add test type router, integration test environment, database/mock support, and API test generation.	Unit + Integration + API prototype
Sprint 8 - Dashboard and Approval	Build risk view, generated test review, assumptions view, verification report, accept/reject/edit, and run history.	Usable web workflow
Sprint 9 - Evaluation and Hardening	Add BugsInPy/Tests4Py adapter, buggy/fixed comparison, metrics, reproducible reports, security review, and final demo.	Evaluation results and final release

Team Distribution - 8 Members

Each owner is responsible for implementation, tests, documentation, and integration with the shared data contracts.

Owner	Area	Responsibilities	Backlog
Member 1	Repository Connector & Git	Clone repositories, branches/commits, GitHub/GitLab integration, patch/branch output.	REPO
Member 2	AST & Repository Analyzer	AST parsing, function/class extraction, project structure, pytest discovery.	ANALYSIS
Member 3	Coverage, Git & Risk	pytest-cov, complexity, Git history, risk formula, ranking explanation.	ANALYSIS/ RISK
Member 4	AI Unit Test Agent	Prompting, context builder, pytest generation, edge cases, test repair.	UNIT/AI
Member 5	Integration & API Agents	Dependency/API detection, mocks, test database, integration and API test generation.	ROUTER/IN TEGRATIO N
Member 6	Sandbox & Execution	Docker sandbox, dependency installation, timeouts, resource limits, logs.	SANDBOX
Member 7	Verification & Mutation	pytest runner, stability, regression, mutation testing, verification report.	VERIFY/MU TATION
Member 8	Backend, Dashboard & Evaluation	FastAPI, PostgreSQL, Redis jobs, dashboard, approval workflow, BugsInPy/Tests4Py evaluation.	API/UI/EVA L

Shared responsibilities

- Everyone writes unit tests for their own component.
- Everyone follows the agreed JSON/data contracts.
- Every feature must have a demo and acceptance criteria.
- No branch is merged without integration testing.
- Member 8 coordinates integration, but every owner contributes to the final demo.

Definition of Done

A backlog item is complete only when all applicable conditions are satisfied:

- Implementation is committed to the team branch.
- Automated tests are added and passing.
- The feature works with a real Python/pytest repository.
- Errors are handled and reported clearly.
- Documentation is updated.
- The feature is connected to the rest of the system.
- A team member other than the owner reviews it.
- The feature is demonstrated during sprint review.

Sprint Review Demo Path

At the end of the project, the team should demonstrate this complete path:

Connect repository → Analyze project → Rank risky function → Generate pytest test → Review assumptions → Run inside Docker → Check stability → Run mutation testing → Show report → Approve test → Create branch/patch

Recommended MVP Cut

If time becomes limited, stop after Sprint 6 and deliver a strong Unit Testing pipeline. Integration tests, API tests, Dashboard enhancements, and benchmark expansion can follow as later increments.